

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2**



Stack dan Queue

Oleh:

Erina Aprilia

NIM. 2510817120020

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Erina Aprilia
NIM : 2510817120020

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
SOAL 1.....	1
A. Source Code	1
B. Pembahasan	2
SOAL 2.....	5
A. Source Code.....	6
B. Output Program.....	7
C. Pembahasan	7
SOAL 3.....	9
A. Source Code	9
B. Pembahasan	10
SOAL 4.....	14
A. Souce Code	15
B. Output Program.....	16
C. Pembahasan	16
TAUTAN GIT	18

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2A.....	7
Gambar 2. Screenshot Hasil Jawaban Soal 2B.....	7
Gambar 3. Screenshot Hasil Jawaban Soal 2C.....	7
Gambar 4. Screenshot Hasil Jawaban Soal 4A.....	16
Gambar 5. Screenshot Hasil Jawaban Soal 4B.....	16
Gambar 6. Screenshot Hasil Jawaban Soal 4C.....	16

DAFTAR TABEL

Tabel 1. Source Code stack.cpp	1
Tabel 2. Source Code prob1.cpp	6
Tabel 3. Source Code queue.cpp	9
Tabel 4. Source Code prob2.cpp	15

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1. Source Code `stack.cpp`

1	<code>#include "stack.h"</code>
2	
3	<code>void init(Stack* s) {</code>
4	<code> s->top = s->data - 1;</code>
5	<code>}</code>
6	
7	<code>bool isEmpty(const Stack* s) {</code>
8	<code> return (s->top < s->data);</code>
9	<code>}</code>
10	
11	<code>bool isFull(const Stack* s) {</code>
12	<code> return (s->top == s->data + MAX - 1);</code>
13	<code>}</code>
14	
15	<code>void push(Stack* s, int value) {</code>
16	<code> if (isFull(s)) {</code>
17	<code> throw "Stack penuh";</code>
18	<code> }</code>
19	<code> s->top++;</code>
20	<code> *(s->top) = value;</code>
21	<code>}</code>
22	
23	<code>void pop(Stack* s) {</code>
24	<code> if (isEmpty(s)) {</code>
25	<code> throw "Stack kosong";</code>
26	<code> }</code>
27	<code> s->top--;</code>
28	<code>}</code>
29	
30	<code>// Melihat elemen teratas</code>
31	<code>int peek(const Stack* s) {</code>
32	<code> if (isEmpty(s)) {</code>
33	<code> throw "Stack kosong";</code>
34	<code> }</code>
35	<code> return *(s->top);</code>

B. Pembahasan

Pada Soal 1, struktur data stack dibangun menggunakan array statis sebagai tempat penyimpanan elemen. Stack merupakan struktur data yang mengikuti prinsip Last In First Out (LIFO), di mana elemen yang terakhir dimasukkan akan menjadi elemen pertama yang diambil kembali. Pendekatan ini sering digunakan karena sederhana dan mampu memberikan akses yang cepat terhadap elemen terakhir.

Implementasi stack ini memanfaatkan sebuah pointer bernama top untuk menandai posisi elemen paling atas. Pointer ini tidak langsung diarahkan ke elemen pertama, melainkan ditempatkan satu posisi sebelum awal array. Cara ini digunakan sebagai penanda bahwa stack masih kosong, sehingga tidak diperlukan variabel tambahan untuk menghitung jumlah elemen yang ada.

Seluruh operasi pada stack dilakukan dengan memanipulasi pointer top, sehingga prosesnya menjadi lebih efisien tanpa perlu melakukan pergeseran data di dalam array. Untuk menjaga agar program tetap aman, kondisi seperti stack penuh dan stack kosong ditangani menggunakan exception dengan throw.

Berikut penjelasan dari setiap fungsi yang ada:

1. void init(Stack s)*

Fungsi init digunakan untuk mengatur kondisi awal stack. Pada fungsi ini, pointer top ditempatkan pada posisi sebelum elemen pertama array, yaitu data - 1. Posisi ini menunjukkan bahwa belum ada elemen yang disimpan di dalam stack.

Algoritma: Pointer top diatur ke posisi satu langkah sebelum elemen pertama array, yaitu $s->data - 1$. Posisi ini menunjukkan bahwa stack belum memiliki elemen.

Justifikasi: Kondisi kosong direpresentasikan dengan posisi pointer yang berada di luar batas awal array. Dengan cara ini, pengecekan kondisi kosong dapat dilakukan hanya dengan membandingkan posisi pointer tanpa memerlukan variabel tambahan.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

2. bool isEmpty(const Stack s)*

Fungsi isEmpty digunakan untuk mengecek apakah stack kosong. Pemeriksaan dilakukan dengan melihat apakah posisi top berada di bawah alamat awal array. Jika kondisi tersebut terpenuhi, maka stack tidak memiliki elemen.

Algoritma: Fungsi mengembalikan nilai benar jika posisi top berada di bawah $s->data$. Hal ini menandakan tidak ada elemen dalam stack.

Justifikasi: Karena pada awalnya pointer top berada di data - 1, kondisi ini hanya terjadi saat stack kosong atau setelah semua elemen dihapus melalui operasi pop.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

3. bool isFull(const Stack s)*

Fungsi `isFull` digunakan untuk mengetahui apakah stack telah penuh. Kondisi penuh terjadi apabila pointer top sudah berada di posisi terakhir array, yaitu $\text{data} + \text{MAX} - 1$. Hal ini menandakan bahwa kapasitas stack telah digunakan secara maksimal.

Algoritma: Fungsi mengembalikan nilai benar jika pointer top sudah berada pada posisi terakhir array, yaitu $s \rightarrow \text{data} + \text{MAX} - 1$.

Justifikasi: Karena ukuran array sudah ditentukan, posisi tersebut merupakan batas maksimum penyimpanan. Ketika pointer mencapai titik ini, tidak ada lagi ruang untuk menambahkan elemen baru.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

4. `void push(Stack s, int value)*`

Fungsi `push` digunakan untuk menambahkan elemen ke dalam stack. Proses dimulai dengan pengecekan apakah stack dalam kondisi penuh. Jika penuh, maka akan dilemparkan exception. Jika masih tersedia ruang, pointer top dinaikkan satu posisi, kemudian nilai baru disimpan pada posisi tersebut. Cara ini memastikan bahwa elemen baru selalu berada di bagian paling atas.

Algoritma: Fungsi diawali dengan pengecekan apakah stack dalam kondisi penuh. Jika penuh, maka akan terjadi exception. Jika tidak, pointer top dinaikkan satu posisi, kemudian nilai `value` disimpan pada alamat yang ditunjuk oleh pointer tersebut.

Justifikasi: Kenaikan pointer sebelum penyimpanan memastikan bahwa elemen baru tidak menimpa data sebelumnya. Dengan cara ini, elemen selalu ditambahkan di bagian atas stack sesuai prinsip LIFO.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

5. `void pop(Stack s)*`

Fungsi `pop` digunakan untuk menghapus elemen teratas dari stack. Jika stack kosong, maka akan terjadi exception. Jika tidak, pointer top diturunkan satu posisi. Penurunan ini sudah cukup untuk menghilangkan akses terhadap elemen teratas tanpa perlu menghapus data secara langsung.

Algoritma: Fungsi ini dimulai dengan pengecekan apakah stack kosong. Jika kosong, maka akan terjadi exception. Jika tidak, pointer top diturunkan satu posisi.

Justifikasi: Penurunan pointer sudah cukup untuk menghilangkan akses terhadap elemen teratas. Data tidak perlu dihapus secara fisik karena elemen di luar jangkauan pointer dianggap tidak valid.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

6. `int peek(const Stack s)*`

Fungsi `peek` digunakan untuk melihat elemen teratas tanpa menghapusnya. Jika stack kosong, maka akan terjadi exception. Jika tidak, nilai yang ditunjuk oleh pointer top akan dikembalikan.

Algoritma: Fungsi ini mengecek terlebih dahulu apakah stack kosong. Jika kosong, maka akan terjadi exception. Jika tidak, nilai pada posisi yang ditunjuk oleh top dikembalikan.

Justifikasi: Pointer top selalu menunjuk ke elemen terakhir yang dimasukkan, sehingga nilai yang dikembalikan merupakan elemen teratas tanpa mengubah isi stack.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

Seluruh operasi pada stack ini memiliki kompleksitas waktu $O(1)$, karena hanya melibatkan perubahan posisi pointer tanpa perulangan atau pencarian data. Hal ini membuat setiap operasi dapat dilakukan dengan sangat cepat.

Dari sisi ruang, tidak ada penggunaan memori tambahan selain array yang telah disediakan di awal, sehingga kompleksitas ruang juga $O(1)$. Pendekatan ini membuat implementasi menjadi sederhana namun tetap efisien.

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.
- Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
 - bilangan bulat
 - operator: +, -, *, /

Format Input

N

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2. Source Code prob1.cpp

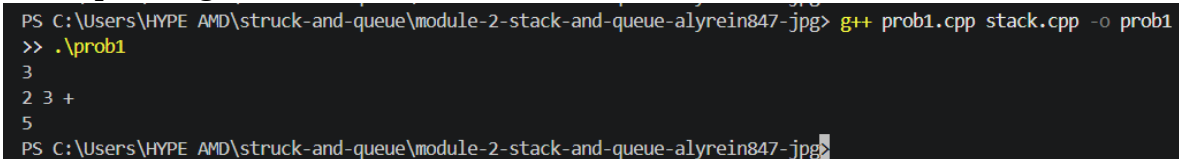
```
1  #include <iostream>
2  #include <sstream>
3  #include "stack.h"
4  using namespace std;
5
6  int main() {
7      Stack s;
8      init(&s);
9
10     int n;
11     cin >> n;
12     cin.ignore();
13
14     string input;
15     getline(cin, input);
16
17     stringstream ss(input);
18     string token;
19
20     while (ss >> token) {
21         if (token == "+" || token == "-" || token == "*"
22 || token == "/") {
23             int b = peek(&s); pop(&s);
24             int a = peek(&s); pop(&s);
25
26             int hasil;
27             if (token == "+") hasil = a + b;
28             else if (token == "-") hasil = a - b;
29             else if (token == "*") hasil = a * b;
30             else if (token == "/") hasil = a / b;
```

```

31         push(&s, hasil);
32     }
33     else {
34         int angka = stoi(token);
35         push(&s, angka);
36     }
37 }
38
39 cout << peek(&s) << endl;
40
41 return 0;
42 }

```

B. Output Program

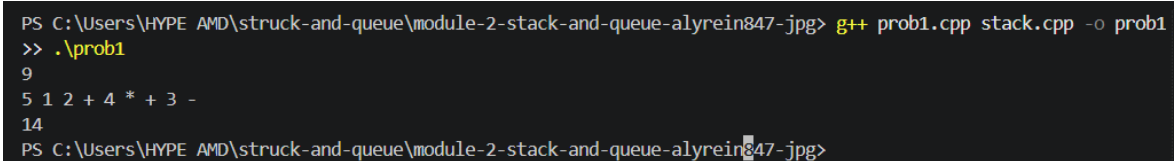


```

PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> g++ prob1.cpp stack.cpp -o prob1
>> .\prob1
3
2 3 +
5
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg>

```

Gambar 1. Screenshot Hasil Jawaban Soal 2A

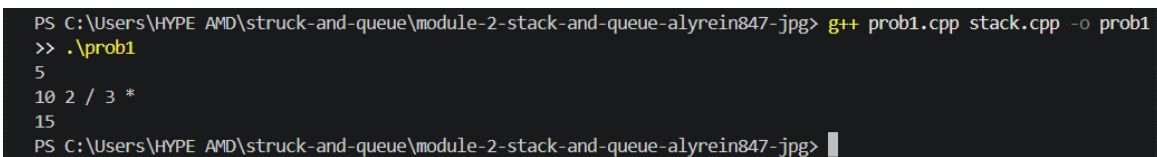


```

PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> g++ prob1.cpp stack.cpp -o prob1
>> .\prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg>

```

Gambar 2. Screenshot Hasil Jawaban Soal 2B



```

PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> g++ prob1.cpp stack.cpp -o prob1
>> .\prob1
5
10 2 / 3 *
15
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg>

```

Gambar 3. Screenshot Hasil Jawaban Soal 2C

C. Pembahasan

Program pada soal ini terdiri atas dua bagian utama, yaitu berkas `stack.h` beserta implementasinya dan berkas program utama. Program ini digunakan untuk menghitung hasil dari ekspresi aritmatika yang diberikan dalam bentuk postfix (Reverse Polish Notation), yaitu format penulisan di mana operator diletakkan setelah operand.

Pada berkas program utama, pertama-tama dilakukan pemanggilan library yang diperlukan, yaitu `iostream` untuk input-output, `sstream` untuk memproses string, serta `stack.h` yang berisi implementasi struktur data `stack`. Selanjutnya, pada fungsi `main()`, dibuat sebuah objek `stack s` dan diinisialisasi menggunakan fungsi `init(&s)`, yang bertujuan untuk mengatur kondisi awal `stack` agar berada dalam keadaan kosong.

Program kemudian membaca nilai `n` yang menyatakan jumlah simbol, diikuti dengan pembacaan satu baris input menggunakan `getline(cin, input)` yang berisi seluruh simbol

dalam bentuk string. Untuk mempermudah pemrosesan, string tersebut diubah menjadi objek stringstream melalui stringstream ss(input), sehingga setiap simbol dapat diambil satu per satu menggunakan perulangan while (ss >> token).

Di dalam perulangan tersebut, setiap token diperiksa apakah merupakan operator atau bilangan. Pemeriksaan dilakukan melalui kondisi if (token == "+" || token == "-" || token == "*" || token == "/"). Jika token merupakan operator, maka program akan mengambil dua nilai teratas dari stack menggunakan kombinasi fungsi peek(&s) dan pop(&s), yaitu melalui baris int b = peek(&s); pop(&s); dan int a = peek(&s); pop(&s);. Urutan pengambilan ini penting karena nilai yang diambil pertama menjadi operand kedua, sedangkan nilai berikutnya menjadi operand pertama.

Setelah kedua nilai diperoleh, program melakukan operasi sesuai operator pada bagian if (token == "+") hasil = a + b; dan seterusnya untuk operator lain. Hasil dari operasi tersebut kemudian dimasukkan kembali ke dalam stack menggunakan fungsi push(&s, hasil), sehingga dapat digunakan pada operasi berikutnya.

Jika token yang dibaca bukan operator, maka token tersebut dianggap sebagai bilangan. Nilai ini dikonversi dari string ke integer menggunakan fungsi stoi(token), kemudian dimasukkan ke dalam stack melalui push(&s, angka). Dengan cara ini, seluruh bilangan akan tersimpan terlebih dahulu sebelum dilakukan operasi.

Setelah seluruh token selesai diproses dalam perulangan, akan tersisa satu nilai di dalam stack. Nilai ini merupakan hasil akhir dari seluruh perhitungan, yang kemudian ditampilkan melalui perintah cout << peek(&s) << endl;.

Justifikasi kebenaran algoritma ini terletak pada penggunaan struktur data stack yang sesuai dengan prinsip Last In First Out (LIFO). Dalam evaluasi ekspresi postfix, dua operand terakhir yang dimasukkan memang harus diambil terlebih dahulu saat operator ditemukan. Dengan demikian, penggunaan stack menjamin bahwa urutan operasi dilakukan dengan benar tanpa memerlukan aturan prioritas operator atau tanda kurung.

Dari sisi kompleksitas, program ini memiliki kompleksitas waktu $O(n)$, di mana n adalah jumlah simbol yang diproses. Setiap token hanya dibaca satu kali, dan setiap operasi stack seperti push, pop, dan peek dilakukan dalam waktu konstan $O(1)$. Oleh karena itu, keseluruhan proses tetap efisien. Dari sisi ruang, kompleksitas yang digunakan adalah $O(n)$, karena dalam kondisi terburuk seluruh simbol berupa bilangan sehingga semuanya disimpan dalam stack. Pendekatan ini membuat program sederhana, efisien, dan sesuai dengan cara kerja mesin yang dijelaskan pada soal.

SOAL 3

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3. Source Code `queue.cpp`

1	<code>#include "queue.h"</code>
2	<code>#include <stdexcept></code>
3	
4	<code>void init(Queue* q) {</code>
5	<code> q->front = q->data;</code>
6	<code> q->rear = q->data - 1;</code>
7	<code>}</code>
8	
9	<code>bool isEmpty(const Queue* q) {</code>
10	<code> return q->rear < q->front;</code>
11	<code>}</code>
12	
13	<code>bool isFull(const Queue* q) {</code>
14	<code> return (q->rear - q->front + 1) == MAX;</code>
15	<code>}</code>
16	
17	<code>void enqueue(Queue* q, int value) {</code>
18	<code> if (isFull(q)) {</code>
19	<code> throw std::overflow_error("Queue penuh");</code>
20	<code> }</code>
21	
22	<code> if (q->rear == q->data + MAX - 1) {</code>
23	<code> int size = q->rear - q->front + 1;</code>
24	<code> for (int i = 0; i < size; i++) {</code>
25	<code> q->data[i] = *(q->front + i);</code>
26	<code> }</code>
27	<code> q->front = q->data;</code>
28	<code> q->rear = q->data + size - 1;</code>
29	<code> }</code>
30	
31	<code> q->rear++;</code>
32	<code> *(q->rear) = value;</code>
33	<code>}</code>
34	
35	<code>void dequeue(Queue* q) {</code>
36	<code> if (isEmpty(q)) {</code>

```

37         throw std::underflow_error("Queue kosong");
38     }
39
40     q->front++;
41
42     if (isEmpty(q)) {
43         q->front = q->data;
44         q->rear  = q->data - 1;
45     }
46 }
47
48 int front(const Queue* q) {
49     if (isEmpty(q)) {
50         throw std::underflow_error("Queue kosong");
51     }
52     return *(q->front);
53 }
54
56 int back(const Queue* q) {
57     if (isEmpty(q)) {
58         throw std::underflow_error("Queue kosong");
59     }
60     return *(q->rear);
61 }

```

B. Pembahasan

Pada penyelesaian Soal 3, digunakan struktur data antrean (queue) yang dibangun menggunakan array statis sebagai media penyimpanan. Struktur ini memanfaatkan dua buah pointer, yaitu front dan rear, untuk mengatur posisi elemen di dalam antrean.

Queue bekerja dengan prinsip First In First Out (FIFO), di mana elemen yang pertama dimasukkan akan menjadi elemen pertama yang dikeluarkan. Dalam implementasinya, elemen baru ditambahkan di bagian belakang melalui pointer rear, sedangkan penghapusan elemen dilakukan dari bagian depan melalui pointer front.

Pada kondisi awal, pointer front diarahkan ke awal array (data), sedangkan pointer rear berada satu posisi sebelum array (data - 1) sebagai penanda bahwa antrean masih kosong. Seluruh operasi dilakukan dengan memanipulasi kedua pointer tersebut, sehingga prosesnya efisien tanpa memerlukan pergeseran data pada setiap operasi.

Definisi struktur queue terdapat pada berkas queue.h, yang berisi deklarasi array sebagai tempat penyimpanan data serta dua pointer untuk menandai posisi awal dan akhir antrean. Selain itu, digunakan mekanisme include guard untuk menghindari duplikasi deklarasi saat proses kompilasi berlangsung.

Seluruh fungsi yang mengatur operasi antrean diimplementasikan dalam berkas queue.cpp. Operasi yang tersedia meliputi penambahan elemen, penghapusan elemen, pengecekan kondisi kosong dan penuh, serta pengambilan elemen terdepan dan terakhir.

Untuk menjaga agar program tetap aman, kondisi tidak valid seperti antrean kosong atau penuh ditangani menggunakan exception handling.

Untuk menjaga keamanan program, kondisi antrean penuh (overflow) dan kosong (underflow) ditangani menggunakan exception handling.

Berikut adalah penjelasan dari setiap fungsi yang digunakan:

1. void init(Queue* q)

Fungsi init digunakan untuk mengatur kondisi awal queue. Pada fungsi ini, pointer front ditempatkan pada awal array, yaitu data, sedangkan pointer rear berada pada posisi sebelum elemen pertama array, yaitu data - 1. Posisi ini menunjukkan bahwa queue masih kosong.

Algoritma: Pointer front diatur ke $q \rightarrow \text{data}$, dan pointer rear diatur ke $q \rightarrow \text{data} - 1$.

Justifikasi: Kondisi kosong direpresentasikan ketika $\text{rear} < \text{front}$, sehingga tidak diperlukan variabel tambahan untuk menghitung jumlah elemen.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

2. bool isEmpty(const Queue* q)

Fungsi isEmpty digunakan untuk mengecek apakah queue kosong. Pemeriksaan dilakukan dengan melihat apakah posisi pointer rear berada di belakang front. Jika kondisi tersebut terpenuhi, maka queue tidak memiliki elemen.

Algoritma: Fungsi mengembalikan nilai benar jika $q \rightarrow \text{rear} < q \rightarrow \text{front}$.

Justifikasi: Kondisi ini hanya terjadi saat queue belum berisi elemen atau setelah semua elemen dihapus melalui operasi dequeue.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

3. bool isFull(const Queue* q)

Fungsi isFull digunakan untuk mengetahui apakah queue telah penuh. Kondisi penuh terjadi apabila jumlah elemen yang tersimpan telah mencapai kapasitas maksimum array.

Algoritma: Fungsi mengembalikan nilai benar jika $(q \rightarrow \text{rear} - q \rightarrow \text{front} + 1) == \text{MAX}$.

Justifikasi: Perhitungan tersebut menunjukkan jumlah elemen dalam queue. Jika sama dengan kapasitas maksimum, maka tidak ada ruang untuk menambahkan elemen baru.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

4. void enqueue(Queue* q, int value)

Fungsi enqueue digunakan untuk menambahkan elemen ke dalam queue. Proses dimulai dengan pengecekan apakah queue dalam kondisi penuh. Jika penuh, maka akan terjadi exception. Jika pointer rear sudah berada di akhir array, maka dilakukan pergeseran elemen ke awal array. Setelah itu, pointer rear dinaikkan satu posisi dan nilai baru disimpan pada posisi tersebut.

Algoritma: Fungsi diawali dengan pengecekan apakah queue dalam kondisi penuh. Jika penuh, maka akan terjadi exception. Jika tidak, dan rear berada di posisi akhir array, maka elemen yang masih valid digeser ke awal array, kemudian pointer front dan rear diperbarui. Setelah itu, pointer rear dinaikkan satu posisi, lalu nilai value disimpan menggunakan $*(q \rightarrow \text{rear}) = \text{value}$.

Justifikasi: Pergeseran dilakukan untuk memanfaatkan kembali ruang kosong di bagian depan array. Kenaikan pointer memastikan elemen baru ditambahkan di bagian belakang sesuai prinsip FIFO.

Kompleksitas Waktu: Rata-rata $O(1)$, terburuk $O(n)$ saat terjadi pergeseran. Kompleksitas Ruang: $O(1)$.

5. void dequeue(Queue* q)

Fungsi dequeue digunakan untuk menghapus elemen paling depan dari queue. Jika queue kosong, maka akan terjadi exception. Jika tidak, pointer front dinaikkan satu posisi. Apabila setelah itu queue menjadi kosong, maka pointer front dan rear dikembalikan ke kondisi awal. Algoritma: Fungsi ini dimulai dengan pengecekan apakah queue kosong. Jika kosong, maka akan terjadi exception. Jika tidak, pointer front dinaikkan satu posisi. Jika setelah itu kondisi queue kosong, maka pointer front dan rear diatur kembali ke posisi awal.

Justifikasi: Kenaikan pointer sudah cukup untuk menghilangkan akses terhadap elemen terdepan tanpa perlu menghapus data secara fisik. Reset dilakukan agar kondisi queue kembali konsisten.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

6. int front(const Queue* q)

Fungsi front digunakan untuk melihat elemen paling depan tanpa menghapusnya. Jika queue kosong, maka akan terjadi exception. Jika tidak, nilai yang ditunjuk oleh pointer front akan dikembalikan.

Algoritma: Fungsi ini mengecek terlebih dahulu apakah queue kosong. Jika kosong, maka akan terjadi exception. Jika tidak, nilai pada posisi yang ditunjuk oleh front dikembalikan menggunakan dereference pointer.

Justifikasi: Pointer front selalu menunjuk ke elemen pertama dalam queue, sehingga nilai yang dikembalikan merupakan elemen terdepan.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

7. int back(const Queue* q)

Fungsi back digunakan untuk melihat elemen paling belakang tanpa menghapusnya. Jika queue kosong, maka akan terjadi exception. Jika tidak, nilai yang ditunjuk oleh pointer rear akan dikembalikan.

Algoritma: Fungsi ini mengecek terlebih dahulu apakah queue kosong. Jika kosong, maka akan terjadi exception. Jika tidak, nilai pada posisi yang ditunjuk oleh rear dikembalikan menggunakan dereference pointer.

Justifikasi: Pointer rear selalu menunjuk ke elemen terakhir dalam queue, sehingga nilai yang dikembalikan merupakan elemen paling belakang.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

Sebagian besar operasi pada queue ini memiliki kompleksitas waktu $O(1)$, karena hanya melibatkan perubahan posisi pointer tanpa proses perulangan. Namun, pada kondisi tertentu saat terjadi pergeseran elemen dalam proses enqueue, kompleksitas waktu dapat

menjadi $O(n)$. Meskipun demikian, kondisi ini tidak terjadi pada setiap operasi, sehingga secara umum performa tetap efisien.

Kompleksitas ruang dari implementasi ini tetap $O(1)$ karena tidak menggunakan memori tambahan selain array yang telah disediakan.

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu. Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih. Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 4. Source Code prob2.cpp

```
1  #include <iostream>
2  #include <sstream>
3  #include "stack.h"
4  using namespace std;
5
6  int main() {
7      Stack s;
8      init(&s);
9
10     int n;
11     cin >> n;
12     cin.ignore();
13
14     string input;
15     getline(cin, input);
16
17     stringstream ss(input);
18     string token;
19
20     while (ss >> token) {
21         if (token == "+" || token == "-" || token == "*"
22         || token == "/") {
23             int b = peek(&s); pop(&s);
24             int a = peek(&s); pop(&s);
25
26             int hasil;
27             if (token == "+") hasil = a + b;
28             else if (token == "-") hasil = a - b;
29             else if (token == "*") hasil = a * b;
30             else if (token == "/") hasil = a / b;
31
32             push(&s, hasil);
33         }
34         else {
35             int angka = stoi(token);
36             push(&s, angka);
37         }
38     }
39     cout << peek(&s) << endl;
40
41     return 0;
42 }
```

B. Output Program

```
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> g++ prob2.cpp queue.cpp -o prob2
>> .\prob2
5 3
1 2 3 4 5
6 9 12
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> |
```

Gambar 4. Screenshot Hasil Jawaban Soal 4A

```
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> g++ prob2.cpp queue.cpp -o prob2
>> .\prob2
5 5
1 2 3 4 5
15
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> |
```

Gambar 5. Screenshot Hasil Jawaban Soal 4B

```
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> g++ prob2.cpp queue.cpp -o prob2
>> .\prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS C:\Users\HYPE AMD\struck-and-queue\module-2-stack-and-queue-alyrein847-jpg> |
```

Gambar 6. Screenshot Hasil Jawaban Soal 4C

C. Pembahasan

Program pada soal ini terdiri atas penggunaan struktur data queue yang didefinisikan dalam berkas queue.h serta program utama yang berfungsi untuk menghitung jumlah elemen pada setiap jendela berukuran k secara berurutan. Permasalahan ini dikenal sebagai sliding window sum, yaitu teknik menghitung jumlah elemen dalam suatu kelompok yang digeser satu per satu sepanjang array.

Pada fungsi main(), program diawali dengan membaca dua buah input, yaitu n sebagai jumlah elemen dan k sebagai ukuran jendela melalui baris `cin >> n >> k;`. Selanjutnya, seluruh elemen array dibaca menggunakan perulangan `for (int i = 0; i < n; i++) cin >> arr[i];` dan disimpan dalam array `arr`.

Program kemudian membuat sebuah objek queue `q` dan menginisialisasinya dengan `init(&q)`. Queue ini digunakan untuk merepresentasikan jendela yang sedang aktif, di mana elemen paling depan menunjukkan elemen yang akan keluar saat jendela digeser, dan elemen paling belakang menunjukkan elemen terbaru yang masuk.

Langkah berikutnya adalah membentuk jendela awal dengan memasukkan k elemen pertama ke dalam queue melalui perulangan `for (int i = 0; i < k; i++)`. Pada bagian ini, setiap elemen dimasukkan menggunakan `enqueue(&q, arr[i]);` dan sekaligus dijumlahkan ke dalam variabel `sum` melalui `sum += arr[i];`. Setelah jendela awal terbentuk, nilai `sum` langsung dicetak menggunakan `cout << sum;`

Setelah itu, program memasuki proses pergeseran jendela melalui perulangan `for (int i = k; i < n; i++)`. Pada setiap iterasi, elemen paling depan dari queue diambil menggunakan `front(&q)` dan dikurangkan dari `sum` melalui baris `sum -= front(&q);`, kemudian elemen

tersebut dikeluarkan dari queue menggunakan `dequeue(&q)`; Langkah ini merepresentasikan elemen yang keluar dari jendela.

Selanjutnya, elemen baru dari array dimasukkan ke dalam queue menggunakan `enqueue(&q, arr[i])`; dan nilainya ditambahkan ke sum melalui `sum += arr[i]`; Dengan cara ini, variabel sum selalu menyimpan jumlah dari k elemen yang sedang berada dalam jendela. Nilai sum kemudian dicetak menggunakan `cout << " " << sum;`, sehingga hasil tiap jendela ditampilkan dalam satu baris dipisahkan spasi.

Justifikasi kebenaran algoritma ini terletak pada penggunaan queue yang sesuai dengan prinsip First In First Out (FIFO). Elemen yang pertama masuk ke dalam jendela akan menjadi elemen pertama yang keluar ketika jendela digeser. Hal ini menjamin bahwa queue selalu merepresentasikan isi jendela yang aktif secara tepat. Selain itu, variabel sum selalu diperbarui dengan mengurangi elemen yang keluar dan menambahkan elemen yang masuk, sehingga tidak perlu menghitung ulang jumlah dari awal setiap kali jendela bergeser.

Dari sisi kompleksitas, program ini memiliki kompleksitas waktu $O(n)$, karena setiap elemen hanya diproses satu kali saat dimasukkan dan satu kali saat dikeluarkan dari queue. Operasi queue seperti `enqueue`, `dequeue`, dan `front` berjalan dalam waktu konstan $O(1)$, sehingga keseluruhan proses tetap efisien. Dari sisi ruang, kompleksitasnya adalah $O(k)$, karena queue hanya menyimpan elemen-elemen dalam jendela aktif yang berjumlah maksimal k elemen.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-2-stack-and-queue-alyrein847-jpg>